# pytoy

A Python toolbox for the Toy CPU

A Python assembler, simulator, and GUI debugger
for Jim Hall's minimal 8-bit educational processor

Manfred Scheucher

# Contents


1 Concepts: how the Toy CPU works ..... 4
1.1 What is a CPU? ..... 4
1.2 The three things you need to know about ..... 4
1.3 The cycle: fetch, decode, execute ..... 4
1.4 One-byte and two-byte instructions ..... 5
1.5 What's missing (on purpose) ..... 5
1.6 The ladder: from C down to the lights ..... 5
2 Instruction set ..... 6
2.1 How the CPU knows the length ..... 6
2.2 Notes on individual instructions ..... 6
2.3 Building bigger operations ..... 6
3 Assembly syntax ..... 7
3.1 The shape of a program ..... 7
3.2 Elements ..... 7
3.3 Number formats ..... 7
3.4 How pytoy lays it out in memory ..... 7
4 Writing programs ..... 9
4.1 Straight-line: `a + b + c` ..... 9
4.2 Loops and counters: multiply by repeated addition ..... 9
4.3 Conditionals from `ifzero` + `xor` ..... 10
4.4 Advanced: self-modifying code for arrays ..... 10
4.5 Tips for writing your own ..... 10
5 Using pytoy ..... 11
5.1 Requirements ..... 11
5.2 GUI (default) ..... 11
5.3 CLI ..... 13
5.4 Bytecode files and minisim ..... 13
5.5 Memory limits and code-overwrite detection ..... 14
6 The C → assembly → CPU pipeline ..... 15
6.1 The stages ..... 15
6.2 Running the pipeline by hand ..... 17
6.3 How the pipeline is tested ..... 17
7 A stack, and recursion ..... 18
7.1 The one primitive you need: indirect access ..... 18
7.2 The stack ..... 18
7.3 Calling convention ..... 18
7.4 Recursive Fibonacci ..... 18
7.5 The compiler emits this automatically ..... 19
8 Memory-mapped I/O ..... 20
8.1 The idea: some addresses are not really memory ..... 20
8.2 The map: `io0`..`io14` and `ready` ..... 20
8.3 Busy-poll: waiting without interrupts ..... 21
8.4 I/O is a GUI feature ..... 21
8.5 Output: `hello_world` ..... 21
8.6 Input: `greet_name` ..... 22
8.7 The rest of the examples ..... 22
8.8 I/O from C: `read` and `write` ..... 23
9 About the Toy CPU ..... 24
9.1 The "switches and lights" heritage ..... 24

9.2 Why pytoy exists ..... 24
9.3 The machine ..... 25
9.4 Building the original (and why v2 on macOS) ..... 25
9.5 What comes from toycpu, and what is original ..... 25
9.6 Credits and license ..... 25

# 1 Concepts: how the Toy CPU works

If you have never written machine language before, this chapter is the foundation. Everything else builds on it.

## 1.1 What is a CPU?

A CPU (central processing unit) is a machine that does one thing over and over, extremely fast: it reads a number from memory, treats that number as an *instruction*, and does what the instruction says. Then it reads the next number, and the next, forever — until an instruction tells it to stop.

That is the whole idea. There is no magic. A program is just a list of numbers sitting in memory. The CPU walks through them one at a time. The Toy CPU is a deliberately tiny version of this idea, small enough to hold in your head all at once.

## 1.2 The three things you need to know about

The Toy CPU has exactly three pieces of state. That's it.

### 1.2.1 Memory — 256 bytes

Memory is a row of **256 numbered boxes**, addressed `0` to `255`. Each box ("byte") holds one number from `0` to `255`.

```
address:   0    1    2    3    4    5   ...  255
value:   [ 20 ][ 7 ][ 22 ][ 8 ][ 0 ][ 5 ] ...  [ 0 ]
```

Crucially, **memory holds both your program and your data.** There is no separate "code" and "data" — it's all just numbers in boxes. The CPU can't tell them apart on its own; *you* decide the layout, and the CPU trusts you.

### 1.2.2 The accumulator (ACC)

The accumulator is a single box *inside* the CPU where arithmetic happens. It holds 8 bits (a value `0`–`255`). Almost every operation flows through it: to add two numbers you `load` one into the accumulator, then `add` the other; to save a result you `store` the accumulator back into a memory box. Think of the accumulator as your hands — you can only hold one value at a time, and you have to put it down (store it) before picking up another.

### 1.2.3 The program counter (PC)

The program counter is the CPU's "you are here" marker. It holds the address of the *next* instruction to run. It starts at `0`, so every Toy CPU program begins at memory address 0. After running an instruction the PC moves forward; `goto` and `ifzero` change it on purpose — that's how loops and branches work.

## 1.3 The cycle: fetch, decode, execute

Everything the CPU does is this loop, repeated:

1. **Fetch** — read the byte at the address in the PC.
2. **Decode** — figure out which instruction that byte means (its *opcode*).
3. **Execute** — do it. If the instruction needs an address (like `add`), the CPU also reads the *next* byte to know which memory box to use.
4. **Advance** — move the PC forward, and repeat.

This continues until the CPU fetches a `stop` instruction.

## 1.4 One-byte and two-byte instructions

Some instructions are self-contained and take **one byte** (`stop`, `right`, `left`, `not`, `nop`). Others need to know *which memory box* to work with, so they take **two bytes**: the opcode, followed by the address.

```
load  a     ->  two bytes:  [ 20 ][ 7 ]     "load, from address 7"
right       ->  one byte:   [ 1 ]           "shift the accumulator right"
```

The CPU knows which is which from the opcode itself (a specific bit marks two-byte instructions — see the instruction set chapter).

## 1.5 What's missing (on purpose)

Compared to a real CPU, the Toy CPU leaves out almost everything: no multiply/divide, no multiple registers, no stack, no interrupts, no overflow flags. Values simply wrap around modulo 256. This is a feature — with so few parts you can understand *the entire machine*, and build up multiplication, comparison, and loops yourself from the primitives.

Modern CPUs are built specifically around a stack plus a base/frame pointer. The Toy CPU has none of that in hardware, but you can simulate one in software: the C compiler builds a stack itself, which is what makes recursion work (see the stack-and-recursion chapter).

## 1.6 The ladder: from C down to the lights

On the original Toy CPU you enter a program the hard way: one byte at a time, in binary, by flipping switches and reading LEDs. That *is* the lesson — with no tools, **you are the assembler**, translating in your head. It teaches you what the machine really does, but it is slow and error-prone for anything longer than a few instructions.

pytoy adds layers on top that make programs easier to write, without ever hiding what the machine executes:

```
C source        (toycc)        int a = 7; return a * b;   most human
    |  compile
    v
assembly        (.toys)        load a / add b / stop      readable, named
    |  assemble  (1:1)
    v
machine code    (bytes)        [20][7][22][8][0]          what the CPU runs
    |  run
    v
Toy CPU                        LEDs + switches            the bare machine
```

Two things make this honest rather than a black box:

- **Assembly maps 1:1 to machine code.** Each mnemonic is exactly one opcode byte; a label is just a name for an address. Assembling is a direct substitution — when you step through a program you can see each source line become the very bytes in memory.
- **The C layer is optional.** `toycc` compiles a small subset of C down to assembly (turning `a * b` into a double-and-add loop, since the CPU has no multiply). It sits on top of the machine, not inside it — you can understand the Toy CPU completely without ever writing a line of C.

# 2 Instruction set

The Toy CPU understands 14 instructions. Each has a numeric **opcode** (the byte that identifies it). Instructions that reference memory are two bytes long (opcode + address); the rest are one byte.

| Mnemonic | Dec | Binary | Bytes | Effect |
|---|---|---|---|---|
| `stop` | 0 | `00000000` | 1 | Halt execution |
| `right` | 1 | `00000001` | 1 | `ACC = ACC >> 1` (shift right) |
| `left` | 2 | `00000010` | 1 | `ACC = ACC << 1` (shift left) |
| `not` | 15 | `00001111` | 1 | `ACC = ~ACC` (bitwise NOT) |
| `and` | 17 | `00010001` | 2 | `ACC = ACC & mem[addr]` |
| `or` | 18 | `00010010` | 2 | `ACC = ACC \| mem[addr]` |
| `xor` | 19 | `00010011` | 2 | `ACC = ACC ^ mem[addr]` |
| `load` | 20 | `00010100` | 2 | `ACC = mem[addr]` |
| `store` | 21 | `00010101` | 2 | `mem[addr] = ACC` |
| `add` | 22 | `00010110` | 2 | `ACC = ACC + mem[addr]` |
| `sub` | 23 | `00010111` | 2 | `ACC = ACC - mem[addr]` |
| `goto` | 24 | `00011000` | 2 | `PC = addr` (jump) |
| `ifzero` | 25 | `00011001` | 2 | if `ACC == 0`: `PC = addr` |
| `nop` | 128 | `10000000` | 1 | Do nothing |

Any opcode not in this table is treated as `nop`.

## 2.1 How the CPU knows the length

Look at **bit 4** (value `0x10`, the `1` in `000 1 0000`). If it is set, the instruction reads a second byte as its address. This is the *fetch bit*:

- `load` = `00010100` → bit 4 set → two bytes.
- `right` = `00000001` → bit 4 clear → one byte.

You never set this bit yourself — it's baked into the opcodes. It is just how the CPU decides whether to read an address after the opcode.

## 2.2 Notes on individual instructions

- **Arithmetic wraps modulo 256.** `255 + 1 = 0`, `0 - 1 = 255`. There is no overflow flag. If a result must fit, keep values small.
- **`right` / `left`** shift the bits by one position, filling with `0`. `left` is a quick multiply-by-2; `right` a divide-by-2 (dropping the low bit). Bits shifted off the end are lost.
- **`ifzero`** is the only conditional. Every "if" and every loop exit is ultimately built from `ifzero`.
- **`goto`** is an unconditional jump — used to loop back or skip ahead.
- **`store`** is how results leave the accumulator and become data in memory.

## 2.3 Building bigger operations

The Toy CPU has no multiply, divide, or compare instruction. You build them:

- **Multiply** → repeated `add`, or doubling with `left`.
- **Compare for equality** → `xor` two values; the result is `0` exactly when they're equal, so follow it with `ifzero`.
- **Loop N times** → keep a counter in memory, `sub` one each pass, and `ifzero` to exit.

# 3 Assembly syntax

Programming the original Toy CPU meant flipping switches to enter raw binary (see the Toy CPU chapter). pytoy lets you write the same programs as readable text in a `.toys` file, and it does the tedious part — turning mnemonics and labels into the right bytes at the right addresses.

## 3.1 The shape of a program

```
# comments start with # and run to end of line
label:  load  var       # an instruction, optionally with a label
        add   other
        goto  label

# data usually goes at the end
var:    42              # decimal
flags:  0b00001111      # binary (0b prefix)
mask:   0xFF            # hexadecimal (0x prefix)
bits:   00001111        # bare 8-bit binary is also accepted
```

## 3.2 Elements

**Instructions** are a mnemonic from the instruction set, optionally followed by an operand. Two-byte instructions (`load`, `add`, `goto`, …) take one operand — a label naming a memory box. One-byte instructions (`stop`, `right`, `not`, …) take none.

**Labels** name a memory address. Write `name:` before an instruction or a data value, then use the name anywhere an address is needed:

```
loop:   sub   one       # "loop" names this instruction's address
        ifzero end      # "end" and "one" are labels defined elsewhere
        goto  loop
```

A label is just a human-friendly name for a number. pytoy figures out the actual address and substitutes it, so you never count bytes by hand.

**Data** is a label followed by a value; it reserves one byte holding that value. Because code and data share memory, put your data *after* the `stop` (or after a `goto`) so the CPU never tries to execute it as an instruction.

## 3.3 Number formats

| Form | Example | Meaning |
|---|---|---|
| Decimal | `42` | Base 10 |
| Hexadecimal | `0xFF` | `0x` prefix |
| Binary | `0b00001111` | `0b` prefix |
| Bare 8-bit | `00001111` | Exactly 8 `0`/`1` digits, read as binary |

All values must fit in one byte (`0`–`255`).

## 3.4 How pytoy lays it out in memory

Lines are placed into memory top to bottom, starting at address 0: a one-byte instruction takes one address; a two-byte instruction takes two (opcode, then the resolved address); a data line takes one address holding its value. So this source:

```
        load  a     # addr 0-1
        add   b     # addr 2-3
        stop        # addr 4
a:      10          # addr 5
b:      20          # addr 6
```

compiles to the bytes `20, 5, 22, 6, 0, 10, 20` at addresses 0–6. The label `a` resolves to `5` and `b` to `6`, which is why `load a` becomes `20, 5`.

You can see this layout for any program with the CLI's export option (`--cli --export`) or in the GUI's memory panel.

# 4 Writing programs

This chapter builds up from a straight-line program to loops, conditionals, and a couple of advanced tricks. Each example is in the `examples/` folder — run them and watch them step. Make sure you're comfortable with the concepts first: memory boxes, the accumulator, the program counter, and that every calculation flows through the accumulator.

## 4.1 Straight-line: `a + b + c`

The simplest programs just do one thing after another. Load a value, combine in more values, stop. `times7_doubling.toys` computes `7 * a` with only adds:

```
        load  a         # acc = a
        add   a         # acc = 2a
        store b         # b = 2a
        load  b
        add   b         # acc = 4a
        store c         # c = 4a
        load  a
        add   b
        add   c         # acc = a + 2a + 4a = 7a
        stop

a:      4
b:      0
c:      0
```

The pattern to internalize: **`load` to start a calculation, `add`/`sub` to build it up, `store` to save an intermediate, `stop` when done.** The data (`a`, `b`, `c`) sits after `stop` so it's never executed.

## 4.2 Loops and counters: multiply by repeated addition

The Toy CPU has no multiply instruction, so you build it. Keep a counter in memory, do work each pass, decrement the counter, and jump back until it hits zero. `multiply_repeated.toys` computes `a * b`:

```
        load  a         # acc = a  (used as loop counter)
loop:   ifzero done     # counter == 0 -> exit the loop
        load  sum
        add   b         # sum += b
        store sum
        load  a
        sub   one       # a--
        store a
        goto  loop      # back to the top

done:   load  sum       # result in accumulator
        stop

a:      7
b:      6
sum:    0
one:    1
```

The loop skeleton is worth memorizing — nearly every Toy CPU loop looks like it: `ifzero` at the top checks the exit condition; the body does the work; a counter is decremented with `sub one`; `goto` jumps back to the top.

## 4.3 Conditionals from `ifzero` + `xor`

`ifzero` is the *only* branch instruction, so every decision is phrased as "is this value zero?". Two useful reductions:

- **Equality:** `xor` two values. The result is `0` exactly when they're equal. Follow with `ifzero` to branch on "equal".
- **Countdown to a condition:** `sub` until you reach `0`, then `ifzero`.

`fibonacci_iter.toys` uses the countdown form to run its loop `n` times:

```
        load  n
loop:   ifzero end      # done after n iterations
        ...             # (compute next Fibonacci number)
        load  n
        sub   one
        store n         # n--
        goto  loop
end:    load  a
        stop
```

## 4.4 Advanced: self-modifying code for arrays

The Toy CPU has no index register, so to walk through an array you modify the program itself while it runs — because code and data are the same memory, you can `store` a new address into an instruction's operand byte.

`sum_array.toys` and `max_array.toys` do this. The key trick is writing a `load` as two raw bytes so the address byte is a normal, patchable memory box:

```
iop:    20              # LOAD opcode, as a raw data byte
iarg:   0               # <- the address byte, overwritten each pass
        add   result
```

Each iteration, the program does `load iarg; add one; store iarg` to bump `iarg` to the next array element. It's a genuine machine-language technique — and a good demonstration of *why* code and data sharing memory is powerful and dangerous.

## 4.5 Tips for writing your own

- **Sketch on paper first.** Decide your variables and the loop shape before writing instructions.
- **Put all data after a `stop` or `goto`** so it's never executed.
- **Watch for wraparound.** Values are mod 256; `fib(14) = 377` overflows, so `fibonacci_iter.toys` only works up to `n = 13`.
- **Step through in the GUI** when something's wrong — watching the PC move and the accumulator change makes bugs obvious.

# 5 Using pytoy

One entry point, `run.py`, with subcommands. `run.py asm` is the **assembler**: it turns a `.toys` file into a `.toyo` byte listing and nothing else. `run.py sim` is the **simulator**: it runs a program and either opens a graphical interface (the default) or runs it in the terminal. Point `run.py sim` at a `.toys` and it assembles first; point it at a `.toyo` and it loads and runs it directly.

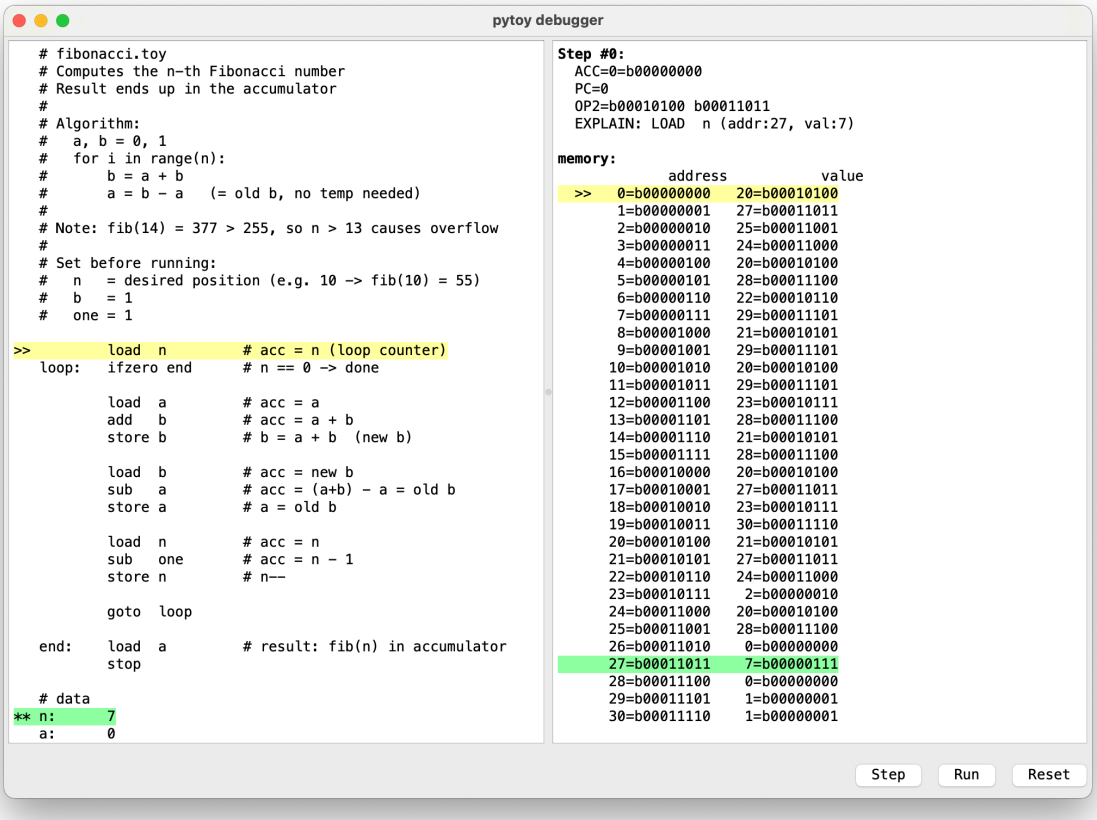



Figure 1: The pytoy GUI: source panel on the left (current instruction highlighted), memory panel on the right showing every byte in binary.

## 5.1 Requirements

- Python 3
- PySide6 — only for the GUI. The CLI works without it.

```
pip install PySide6
```

## 5.2 GUI (default)

Just point the simulator at a program (or run `python3 run.py` with no arguments to open the GUI empty and load an example via the Load button):

```
python3 run.py sim examples/asm/03-programs/fibonacci_iter.toys
```

The window has:

- **Source panel** — your program, with the current instruction highlighted in yellow and any memory it references in green. Each row is prefixed with a display-only line number (right-aligned, like `3 |`) matching the line numbers your text editor shows for the `.toys` file. The numbers

are never written into the source or any saved file — assembly refers to lines by label, so the numbering is just a reading aid.
- **Memory panel** — every address and its value in binary, so you watch the bits change as the program runs.
- **Controls** — Step, Run, and Reset buttons.

| Key | Action |
|---|---|
| `Space` | Step one instruction |
| `R` | Run to completion |
| `Escape` | Reset to the start |

Click any line to highlight it (orange) on both panels — handy for tracing which memory box a label refers to.

**Right-click the source panel** to open an *Edit assembly* window: a plain-text editor prefilled with the current program's assembly source. Change it and press Save — pytoy re-assembles the edited text fresh and reloads the program at step 0, so you can tweak a program and re-run without leaving the GUI. If the edited text has an assembly error, the error is shown and the editor stays open so you can fix it; Cancel discards your changes. (A `.toyo` loaded as bytecode has no assembly source, so right-click there just says there is nothing to edit.)

Once you've edited the assembly this way, Load/Reload/Examples ask before discarding it (the same confirmation that guards hand-edited memory), so a stray Reload can't silently throw your edit away.

### 5.2.1 Loading C — the C → assembly view

The Load button also accepts C sources (`.toyc`), and so does the Examples browser (which lists the C examples grouped per compiler). A C file is never loaded blindly: pytoy compiles it first and opens a **C → assembly window** so you see exactly what the compiler produced before anything runs.

The window shows two panels side by side: your C source on the left and the generated assembly on the right, each with the same line-number gutter as the main source panel. The title says which compiler was used (`minicc` or `toycc`). **Click a line on either side** to highlight it and every matching line on the other: one C statement normally compiles to several assembly lines, so clicking `int x = 5;` lights up its `load`/`store` pair, and clicking an assembly line lights up the C statement it came from. Lines with no counterpart (the generated header, data bytes) simply don't highlight.

Press **Load** to assemble the shown assembly and load it into the simulator at step 0 (Cancel discards it). The compiler records which C line each assembly line came from as a harmless `\# @c:N` comment on the line; the assembler ignores comments, so the loaded program is byte-for-byte what you'd get without them, and the markers stay visible in the assembly panel. If the C source fails to compile, the error is shown and no window opens. For a `.toyc` opened via Load (where you haven't picked a compiler), pytoy tries both and prefers `toycc` — the fuller one — when both accept the file; the Examples browser uses the compiler of the node you picked. Reload remembers that choice: reloading the same `.toyc` reuses the compiler it was loaded with, so a file both compilers accept doesn't silently switch between them.

After you press Load the main window keeps the C level: it shows a third **C source** column to the left of the assembly panel, so you see C, assembly and memory side by side. The columns are adaptive — a `.toyo` shows memory only, a `.toys` shows assembly and memory, and a compiled `.toyc` shows all three.

**Click a line in any column** and the matching lines in the others light up (orange): a C statement lights the assembly lines it compiled to and the memory bytes those occupy; clicking an assembly

line or a memory cell works back up to the C statement it came from. Clicking a blank or unrelated line clears the highlight everywhere. While the program runs, a `store` that overwrites a byte is shown in red not only on the byte but also on the assembly line and the C line it came from, so you can trace a write back to its source.

## 5.3 CLI

Use `-c` / `--cli` to run in the terminal instead of opening the window:

```
python3 run.py sim examples/asm/03-programs/fibonacci_iter.toys --cli
```

By default the CLI runs step-by-step with full verbose output; press Enter to advance each instruction.

| Flag | Effect |
|---|---|
| `-c`, `--cli` | Run in the terminal instead of the GUI |
| `-r`, `--run` | Run all steps without pausing |
| `-q`, `--quiet` | Compact, one line per step |
| `-x`, `--export` | Also write the compiled listing to a `.toyo` file |

Run `python3 run.py sim -h` for the full list.

`sim` expects an assembly (`.toys`) or bytecode (`.toyo`) program. Pointing it at a C source (`.toyc`) stops with a hint to compile it first (`run.py cc …`); the C → assembly view is a GUI feature, reached via the Load/Examples buttons.

```
# Run to the end, compact output:
python3 run.py sim examples/asm/02-extended/multiply_repeated.toys --cli --run --quiet

# Assemble to a standalone .toyo, then run it:
python3 run.py asm examples/asm/03-programs/max_array.toys      # -> examples/asm/03-
programs/max_array.toyo
python3 run.py sim examples/asm/03-programs/max_array.toyo --cli --run
```

The final line reports the result in the accumulator in binary, hex, and decimal.

Three ready-made `.toyo` examples are checked in under `examples/toyo/` (`sum3.toyo`, `fibonacci.toyo` and `multiply_repeated.toyo`), so you can try loading a compiled byte listing without assembling one first. They were generated from the matching assembly sources with `run.py asm`:

```
python3 run.py asm examples/asm/03-programs/sum3.toys      -o examples/toyo/sum3.toyo
python3 run.py asm examples/asm/03-programs/fibonacci_iter.toys -o examples/toyo/
fibonacci.toyo
python3 run.py asm examples/asm/02-extended/multiply_repeated.toys -o examples/toyo/
multiply_repeated.toyo
```

Loading a `.toyo` shows only the memory panel — a compiled listing carries no re-runnable source. (Most `.toyo` files are build artifacts and git-ignored; these three are committed on purpose as examples.)

## 5.4 Bytecode files and minisim

A `.toyo` comes in two formats, and pytoy tells them apart by content. `run.py asm` writes the byte listing described above. Save -> Bytecode in the GUI writes only the values: one integer (0–255) per line, up to the last non-zero cell. The remaining cells are 0, so a program with values in addresses 0

to 6 is 7 lines. You can add comments after a `#`. It holds the program as built plus your hand edits, and nothing a run stored.

`pytoy/minisim.py` is a tiny standalone simulator for such a file, short enough to read in one go. It imports nothing from the rest of pytoy and has no I/O:

```
python3 pytoy/minisim.py tests/fixtures/fibonacci.txt
```

```
step=1 acc=0 pc=0 op=20
...
step=81 acc=13 pc=26 op=0
stopped after 81 steps, acc=13
```

Each line shows the state before that step runs. With `-v` every step also prints the memory, 16 cells per line.

## 5.5 Memory limits and code-overwrite detection

The Toy CPU has exactly 256 bytes, shared by code and data. Two things can go wrong at that boundary, and pytoy handles both:

**Too big to fit.** If a program assembles to more than 256 bytes, the assembler stops with a clear message rather than crashing:

```
ERROR: Program is too big: it needs 402 bytes, but the Toy CPU has only 256.
```

The C compiler `toycc` performs the same check on its generated output, so an oversized C program is caught at compile time instead of failing later.

**Overwriting code.** Because code and data share memory, a `store` can write over your own instructions. That is a legitimate technique (the array examples use it deliberately — see the writing-programs chapter), so it is allowed by default. But when you *don't* mean to do it, it is a nasty bug. The optional `-d` / `--detect-code-overwrite` flag turns on a guard:

```
python3 run.py sim myprog.toys --cli -d
```

The guard uses the `# data` marker in your source — a line whose only content is `# data` — to know where the code region ends and data begins. If a `store` writes below that line (into code), the simulator warns and asks whether to continue; in the GUI it pops up a dialog. If the program has no `# data` marker, the guard can't know the boundary and prints a note that detection is off. The flag is off by default, keeping the bare machine's "anything goes" behaviour unless you opt in.

# 6 The C → assembly → CPU pipeline

pytoy ships with **toycc**, a small compiler that turns a simplified subset of C into Toy CPU assembly (`.toys`). Writing C is more pleasant than hand-writing assembly, and it lets you see, stage by stage, how a high-level program becomes raw machine bytes.

The compiler lives in `pytoy/compiler.py`; the README in `compiler/` documents the exact supported C subset.

## 6.1 The stages

```
program.toyc                              (simplified C source)
    |
    |  toycc  --  lex -> parse -> code generation
    v
program.toys                              (Toy CPU assembly text)
    |
    |  toyasm assembler (two-pass)
    v
256-byte memory image + symbol table      (opcodes & data bytes)
    |
    |  toysim simulator (fetch -> decode -> execute)
    v
final accumulator (ACC)                   (the program's result)
```

| Stage | Tool | Input | Output |
|---|---|---|---|
| 1. Compile | `toycc` | `.toyc` source text | `.toys` assembly text |
| 2. Assemble | toyasm | `.toys` text | memory image + symbols, or errors |
| 3. Simulate | toysim | memory image | final ACC value |

### 6.1.1 Stage 1 — toycc (C → assembly)

toycc is a hand-written compiler (standard library only): a **lexer** splits the source into tokens, a **recursive-descent parser** builds a syntax tree, and a **code generator** walks that tree emitting Toy CPU instructions. Because the machine is so small, code generation makes concrete choices you can inspect in the output:

- Every C variable gets a fixed **data byte** in memory; temporaries get their own bytes too.
- `a * b` becomes a **double-and-add loop** (there is no multiply opcode).
- Comparisons reduce to the one available conditional, `ifzero` — e.g. `a == b` compiles to `a - b` followed by `ifzero`, and ordering uses the sign-bit-of-difference trick.
- `while` / `for` / `if` become `goto` and `ifzero` with generated labels.

### 6.1.2 Functions

toycc supports multiple functions that call each other (see `examples/c/03-programs/sum_of_squares.toyc`). The Toy CPU has no call/return instruction, no stack pointer, and no indirect jump, so each function is compiled with **global slots + marker dispatch**:

- each function `f` gets fixed data bytes — one per parameter, a return-value slot, and a return-marker slot;
- its body is emitted once; `return e` stores `e` and jumps to `f`'s dispatch;
- a call writes the arguments into the parameter slots, sets the marker to a number identifying **this** call site, and jumps to the body;
- a compare-chain on the marker jumps back to the correct call site.

### 6.1.3 Recursion

Because a function has only one set of slots, a naive recursive call would clobber its caller's values. So around **every** call, toycc saves the caller's live values onto a real stack (one `sp` byte plus self-modifying indirect push/pop, growing down from address 255) and restores them afterward — giving each activation its own copies. This makes recursion, including mutual recursion, work automatically; `examples/c/03-programs/fibonacci_rec.toyc` is a recursive Fibonacci. It is the same *idea* as the hand-written `examples/asm/03-programs/fibonacci_rec.toys` (next chapter) — a self-modifying-code stack — though the compiler's stack grows **down** from 255 while the hand-written one grows up; the direction is a free choice, the principle is identical.

Save/restore is inserted automatically, and only around calls whose caller is recursive; non-recursive code gets none (and no stack byte at all). The `--stackfree` flag enforces that no stack is used, erroring if any function recurses. The stack has no bounds check and shares the 256 bytes with code and data, so deep recursion can overflow into data and corrupt it. toycc warns when the **compiled program itself** leaves little room for the stack — but note this is a static size check, not a depth guard: a program that fits with plenty of headroom can still overflow at run time if it recurses deep enough, silently. Keep recursion shallow and inputs small.

### 6.1.4 Arrays

Fixed-size local arrays (`int a[10] = {...};`, `a[i]` read/write with any index expression) are supported. With no index register, `a[i]` compiles with self-modifying code: compute `base + i`, patch it into a raw load/store's address byte, and execute it — the same trick the `examples/asm/03-programs/sum_array.toys` and `examples/asm/03-programs/sort_array_inline.toys` assembly examples use. `examples/c/03-programs/sum_array.toyc`, `examples/c/03-programs/max_array.toyc` and `examples/c/03-programs/sort_array_inline.toyc` operate on the ten digits of pi. (Arrays are not saved across recursive calls, so a recursive function may not declare one — the compiler rejects that.)

### 6.1.5 Pointers

An address on the Toy CPU is just a byte, so a pointer is an ordinary one-byte variable holding an address. toycc supports `int *p`, address-of `&x` (and `&a[i]`), and dereference `*p` as both an rvalue and an assignment target (`*p = e`). `&x` compiles to a data byte initialised to `x`'s address; `*p` read/write reuse the same self-modifying-code trick as array indexing (patch the address from `p` into a raw load/store, then execute it).

Crucially, an array passed to a function **decays to a pointer**: `int f(int a[], int n)` receives the array's address, and `a[i]` inside `f` means `*(a + i)` through that address — so a function can sort or fill the caller's array in place. `examples/c/03-programs/sort_array_fn.toyc` is the bubble sort written as such a function. (Taking the address of a local inside a **recursive** function is rejected, since `&x` names a single shared slot the recursion stack can't follow.)

Because the recursion stack, array indexing and pointer dereference all store into patched instructions, toycc's output uses self-modifying code by design. Don't run it with `--detect-code-overwrite` (previous chapters): that guard flags any store below the data region as overwriting code, so it would false-trip on this legitimate self-modification and halt the program.

### 6.1.6 Stage 2 — the assembler (assembly → bytes)

toyasm's two-pass assembler resolves labels to addresses and emits the 256-byte memory image. This is the same assembler used for hand-written `.toys` programs; toycc's output is nothing special to it. Invalid input yields errors instead of a memory image.

### 6.1.7 Stage 3 — the simulator (bytes → result)

The simulator runs the fetch–decode–execute cycle until `stop`, leaving the program's `return` value in the accumulator, printed as the final `Result: ACC = …` line.

## 6.2 Running the pipeline by hand

```
# Stage 1: compile C to assembly
python3 run.py cc examples/c/02-extended/multiply_operator.toyc

# Stages 2+3: assemble and simulate (the simulator assembles the .toys, then runs it)
python3 run.py sim examples/c/02-extended/multiply_operator.toys --cli --run --quiet
#   -> Result: ACC = 42

# Or all three at once with toycc's --run flag:
python3 run.py cc examples/c/02-extended/multiply_operator.toyc --run
```

## 6.3 How the pipeline is tested

The whole pipeline is exercised by automated tests in `tests/test_toycc.py`, run with `pytest`. The tests are **data-driven**: every `examples/c/*.toyc` file carries a machine-readable `// expect: N` annotation stating its correct result. For each example the harness runs all three stages (`compile_source #sym.arrow assemble #sym.arrow simulate`) and asserts the accumulator equals `N`. Adding a new example automatically adds a test.

Focused unit tests cover individual compiler features (arithmetic, bitwise ops, shifts, `if`/`else`, `while`, `for`, comparisons, wraparound) so a failure points at the specific feature. A git pre-commit hook (installed via `scripts/install-hooks.sh`) runs this suite before each commit.

# 7 A stack, and recursion

The Toy CPU has no stack pointer, no call/return instruction, and no index register. It is tempting to conclude that recursion is impossible on it. It is not — you can build a real call stack by hand, and this chapter shows how. The worked result is `examples/asm/03-programs/fibonacci_rec.toys`, which computes Fibonacci **recursively**.

## 7.1 The one primitive you need: indirect access

The trick is **self-modifying code**, the same technique the `sum` and `max` examples use. A two-byte instruction like `load` is an opcode byte followed by an address byte. If you write those two bytes as data,

```
iop:    20        # LOAD opcode, as a raw byte
iarg:   0         # ← address byte, patched at runtime
```

then storing a value into `iarg` and executing the `iop/iarg` pair loads from **that** address. That is `*p` — indirect read. Indirect write works the same with the STORE opcode. Indirect access is all a stack needs.

## 7.2 The stack

Reserve a block of memory for the stack and keep one byte, `sp`, holding the address of the next free slot.

- **push v**: write `v` to `mem[sp]` (an indirect store through `sp`), then `sp = sp + 1`.
- **pop**: `sp = sp - 1`, then read `mem[sp]` (an indirect load through `sp`).

The one subtlety: `sp` changes on every push and pop, so the indirect instruction's address byte must be **re-patched from the current `sp` before each access**. (This is different from `examples/asm/03-programs/sum_array.toys`, which patches its pointer once and then walks it.)

The stack grows upward; there is no hardware bounds check, so a program must stay within a sensible depth. `--detect-code-overwrite` does not help here (it watches for stores **below** the data region, but this stack grows up), so the safe budget is simply: max depth × frame size ≤ free bytes.

## 7.3 Calling convention

Each call pushes a frame `[return-marker][argument]`. Crucially the return marker lives **on the stack**, one per activation — so a recursive call cannot clobber the outer call's marker. The shared function body:

1. reads its argument from the frame,
2. does its work, pushing and popping its own temporaries (so a recursive call gets its own copies — this is what makes recursion correct),
3. leaves the result in the accumulator,
4. pops the frame and **dispatches** on the return marker: a compare-chain (`load marker; ifzero site0; sub 1; ifzero site1; …`) jumps back to whichever call site invoked it.

## 7.4 Recursive Fibonacci

`fib(n) = n` if `n < 2`, else `fib(n-1) + fib(n-2)`, all mod 256. The recursive body, for the non-base case:

1. compute `n-1` and `n-2` up front (later recursive calls bury the frame, so don't rely on `n` staying reachable);
2. push a return marker and `n-1`, jump to `fib`; on return the accumulator holds `fib(n-1)`. **Push it onto the stack to save it** — the next call must not destroy it;
3. push a return marker and `n-2`, jump to `fib`; on return the accumulator holds `fib(n-2)`;

4. store `fib(n-2)` into a scratch byte (there is only one accumulator), pop the saved `fib(n-1)`, and add the scratch — giving `fib(n-1) + fib(n-2)`;
5. return: pop the frame and dispatch on the marker.

The subtle point — pointed out during design and confirmed by hand-tracing `fib(3) = 2` — is that you must compute and **stash** `fib(n-1)` on the stack before starting `fib(n-2)`, because both use the same shared body and the same stack. `examples/asm/03-programs/fibonacci_rec.toys` is verified against `fib(0..13)`.

## 7.5 The compiler emits this automatically

This hand-written program was the proof of concept; the C compiler `toycc` now generates the same kind of stack machinery on its own. It uses a slightly different but equivalent scheme — instead of hand-picking what to stack, it saves the caller's live values around each call and restores them after (see the recursion section of the previous chapter) — but the core idea is identical: a self-modifying-code stack gives each activation its own copies, so recursion works. `examples/c/03-programs/fibonacci_rec.toyc` is the C version of this very program, compiled and run through the same pipeline.

# 8 Memory-mapped I/O

So far the Toy CPU has been a closed box: 256 bytes of memory, an accumulator, a program counter, and nothing else. A program can compute, but it has no way to reach anything *outside* itself. This chapter adds the one mechanism that lets a CPU talk to the outside world: **memory-mapped I/O**.

## 8.1 The idea: some addresses are not really memory

The trick is almost embarrassingly simple. You pick a few memory addresses and agree that they don't behave like ordinary RAM. Writing to one of those addresses sends a byte *out*; reading from one brings a byte *in*. The CPU core does not change at all. It still just loads and stores bytes. It is the world on the other side of those addresses that does something special with them.

This is not a toy simplification. It is how real CPUs talk to real hardware. A keyboard controller, a disk, a network card, a screen: each one is wired to a range of addresses, and the CPU drives it by reading and writing those addresses with the same `load`/`store` it uses for everything else. There is no separate "I/O instruction set" on most machines. I/O *is* memory access, aimed at an address that happens to have a device behind it instead of a RAM chip.

It is worth saying plainly, because it is easy to assume otherwise: **I/O is not only user input.** Typing characters at a keyboard is just one thing that can sit behind an address. The same door opens onto a disk, a sensor, a second processor, or a much larger memory than the CPU can address directly. The Toy CPU can only name 256 bytes; think of those 256 bytes as a small window, almost a cache, and memory-mapped I/O as the hatch through which a far bigger outside world is passed in and out, a few bytes at a time. The human at the keyboard is one peripheral among many.

## 8.2 The map: `io0`..`io14` and `ready`

pytoy reserves the top of memory for I/O:

| Address | Label | Role |
|---|---|---|
| `240`–`254` | `io0`–`io14` | 15 bidirectional I/O registers |
| `255` | `ready` | a plain register the program polls |

The labels `io0` through `io14` and `ready` are **reserved names in the assembler** — you write `store io0` or `load ready` and the assembler fills in addresses 240 and 255 for you, the same way it resolves any label. They are not special opcodes; `store io0` is an ordinary `store` (see the *Instruction set* chapter) that happens to target address 240.

The 15 `io` cells are **bidirectional**: a program writes them to send a value out, or reads them to take a value in. Which direction they mean is pure convention between the program and whoever is on the other side.

`ready` (address 255) is just as ordinary. It is a single byte the program reads to decide whether input has arrived. The convention is: `0` means *nothing yet*, and a non-zero value means *input is available* — often the non-zero value is the *count* of valid `io` cells. Nothing enforces this. `ready` is a normal byte like any other; the meaning lives entirely in the program and the convention.

One housekeeping consequence: because 240–255 are reserved, the software recursion stack (see *A stack, and recursion*) now starts at address 239 and grows downward, keeping clear of the I/O region.

## 8.3 Busy-poll: waiting without interrupts

The Toy CPU has no interrupts and no blocking "read a byte" instruction. Its only conditional is `ifzero`. So how does a program wait for input to show up? It *busy-polls*: it reads `ready` over and over in a tight loop, and only moves on once `ready` is non-zero.

```
wait:   load  ready       # read the ready flag
        ifzero wait        # still 0? loop back and read again
        ...                # ready != 0 -> input is here, go use it
```

That two-line loop is the whole pattern. While `ready` is `0` the program spins, doing nothing but checking again. The instant `ready` becomes non-zero the `ifzero` falls through and the program reads the `io` cells.

Polling is a trade: it burns CPU cycles for the sake of being dead simple. A real system would usually let the CPU sleep and wake it with an interrupt. But the pattern also generalises nicely — a polling loop could do other work between checks, or give up after a while (a timeout), or take a different branch if the input never comes. The canonical loop above just waits forever.

## 8.4 I/O is a GUI feature

In pytoy, I/O lives in the **GUI**. The simulator's I/O widget shows the 15 `io` cells plus the `ready` register as a 16th cell, all in one row (two rows in binary mode, where each value is eight characters wide). While the program is polling — the current instruction is `load 255` — it highlights `ready` in red so you can see the machine waiting on you. The running program is the device's software; *you* are the device. You click any cell to edit it — the exact same dialog as editing memory in the CPU panel — and the program, spinning in its poll loop, reacts. `ready` can also be set from the optional "set ready" input row, which you enable from the *View* menu.

The headless `--run` (see *Using pytoy*) has no outside world to talk to, so it treats 240–255 as plain memory: writes land there and stay, reads return whatever is there. A program that busy-polls `ready` under `--run` will spin until it hits the step cap, because nothing ever sets `ready`. That is expected; busy-polling only makes sense when something can change `ready` from outside, which is what the GUI provides.

## 8.5 Output: `hello_world`

The simplest direction is output: the program writes bytes to the `io` cells and the widget displays them. `examples/asm/04-io/hello_world.toys` writes the ASCII codes for "Hello world!" into `io0`..`io11`:

```
        load  H
        store io0
        load  e
        store io1
        ...
        stop
H:   72
e:  101
...
```

Each `io` cell holds one character code. Pick *ascii* under *View* → *I/O format* and the bytes read as letters; a non-printable code (a control code, or a high byte) shows as a middle dot `·`, kept distinct from a real `.`. The same bytes in *decimal*, *hex*, or *binary* mode are just different views of the identical memory — the program never changed, only the display did.

## 8.6 Input: `greet_name`

Input combines the busy-poll with reading the `io` cells. `examples/asm/04-io/greet_name.toys` reads a name and writes back a greeting. In the GUI you type one ASCII character per cell (`io0`, `io1`, …), then set `ready` to the *number* of characters you typed. The program waits, then uses that count as the length:

```
wait:   load  ready
        ifzero wait
        load  ready
        store n              # n = number of name characters
```

With `n` in hand it copies the name out of the `io` cells into a scratch area, writes "hallo " into `io0`..`io5`, appends the name after it, and finishes with a `!`. Type `Max` with `ready = 3` and `io0`..`io9` come back as `h a l l o _ M`
`a x !`.

Two details are worth noticing. First, the program must copy the name to scratch *before* writing "hallo ", because the greeting overwrites `io0`..`io5`, which may still hold name characters. Second, it indexes the `io` cells by the runtime count `n` using **self-modifying code** — patching the address byte of a `load`/`store` on each pass — exactly the indirect-access technique from the *A stack, and recursion* chapter. The same machinery drives `examples/asm/04-io/sort_array_io.toys`, which reads `N` values, bubble-sorts them, and writes them back.

## 8.7 The rest of the examples

`examples/asm/04-io/` collects the I/O programs:

| Program | What it shows |
|---|---|
| `numbers` | write `0`..`14` into `io0`..`io14` — the minimal output program (decimal view) |
| `numbers_hex` | same bytes, opens in hex view (`0`..`9`, `a`..`e`) |
| `numbers_ascii` | same bytes, opens in ascii view |
| `hello_world` | write the ASCII of "Hello world!" (output, ascii view) |
| `fibonacci_io` | write the Fibonacci numbers `0`..`233` into the `io` cells |
| `countdown` | count down from 10, writing each step to a cell |
| `poll_demo` | the canonical busy-poll: wait on `ready`, echo `io0` to `io1` |
| `greet_name` | read a name, write "hallo NAME!" (input + output) |
| `sort_array_io` | read `N` values, sort ascending, write them back |
| `pin_check` | read a 4-digit PIN (ascii), check it against 1337; right PIN reveals the secret 42 ("succ"), wrong PIN gives 0 ("fail") |
| `pin_check_hardened` | the same check made *constant-time* (no per-digit branch), to close the timing side channel |

`pin_check` is the running example for a short tour of security basics on the Toy CPU — brute force, reading the bytecode, side channels, and fault injection, and what each countermeasure buys. See `doc/design/pin-security.md`.

Most have a `# pytoy: output=…` hint on the first line that sets the I/O widget's default display format (overridable any time from *View → I/O format*). Several have C counterparts under `examples/c/04-io/` (see the next section).

## 8.8 I/O from C: `read` and `write`

The C compiler (toycc) exposes memory-mapped I/O through two built-in functions, so a C program never has to touch the `io` cells by hand:

| Call | What it does |
|---|---|
| `write(data, k)` | send `data[0]..data[k-1]` out to `io0`, `io1`, … (addresses 240, 241, …) |
| `read(data)` | busy-poll `ready` until it is non-zero, then copy that many values from `io0`.. into `data[0]`.., and return the count |

`read` returns the number of values it read (the `ready` value), so the usual shape is `int k = read(buf);` followed by a loop up to `k`. Both take a buffer that is any address expression — an array name, or pointer arithmetic such as `read(data + 6)` to read into the middle of a buffer.

Character literals make ascii programs readable: `data[0] = 'H';` compiles to the byte 72, so `hello_world.toyc` spells out the greeting instead of writing raw codes. By default a compiled I/O program opens the widget in ascii view; add a `// pytoy: output=decimal` line to override it.

**Limits are part of the lesson.** pytoy is a tool for learning and visualising how a CPU works, not an applied compiler. The whole program — code and data — must fit in 256 bytes (and below address 240, since 240..255 are the I/O region), so anything real runs out of space fast. On top of that every value is an 8-bit int that wraps at 256, the recursion stack is tiny, and there is no overflow or bounds checking anywhere. These are not bugs to fix; they are the machine. The point is to see the moving parts clearly, then reach for a real toolchain when you want to *apply* it.

The I/O built-ins follow the same spirit. `read` and `write` only move the 15 `io` cells, so a count is always 0..15 in practice. The copy loop tests `i < count` with the machine's sign bit, which is exact up to 128 — far more than
15. A larger count is only reachable if the outside world writes a bogus `ready`

value, and the loop then copies nothing. That is left unguarded on purpose: a full unsigned compare would cost code for a case a well-behaved device never produces.

The C I/O examples under `examples/c/04-io/`:

| Program | What it shows |
|---|---|
| `hello_world` | write "Hello world!" from a constant array (output, ascii) |
| `greet_name` | put "Hallo " in a buffer, `read(data + 6)` the name, `write(data, k + 6)` the greeting |
| `pin_check` | `read` a 4-digit PIN, compare to 1337 digit by digit, reveal the secret 42 on success |
| `pin_check_hardened` | the constant-time version: OR the per-digit differences, one final `diff == 0` test |

# 9 About the Toy CPU

pytoy is a Python toolbox (assembler, simulator, and C compiler) for the **Toy CPU**, a minimal educational processor created by Jim Hall for the FreeDOS Project. This chapter records where the machine comes from and how pytoy relates to it.

- Official page: https://jimhall.itch.io/toy-cpu (free, MIT-licensed)
- Source: https://github.com/freedosproject/toycpu
- Jim Hall's talk (VCF East, 50 min): https://www.youtube.com/watch?v=zhoL1ZSjGfM

## 9.1 The "switches and lights" heritage

Early microcomputers like the **Altair 8800** and **IMSAI 8080** (both 1975) had no keyboard or screen. You entered programs by flipping a bank of **switches** to set each binary instruction, and read results from a row of **LEDs** ("lights"). Programming meant knowing the raw opcodes by heart and toggling them in bit by bit.

The Toy CPU faithfully recreates this: a counter, an instruction, and an accumulator, each shown as 8 lights, with a tiny instruction set instead of the real Altair opcodes.

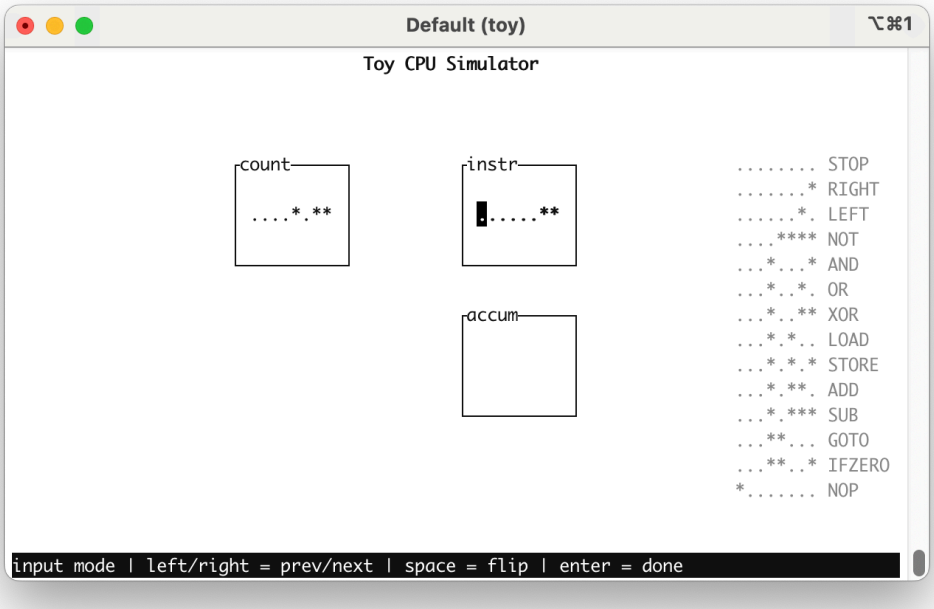



Figure 2: Jim Hall's original Toy CPU (v2, ncurses). The `count`, `instr`, and `accum` boxes are 8-bit LED patterns; the legend on the right lists every opcode. You program it by flipping bits: *left/right = prev/next, space = flip, enter = done*. pytoy exists to spare you exactly this bit-by-bit entry, without hiding what it produces.

## 9.2 Why pytoy exists

I started pytoy because I find the Toy CPU very elegant. It shows clearly how a CPU works, with few enough parts that the whole machine fits in your head. Working at the assembly level makes it far more accessible than entering bits by hand, and the Toy CPU is really just bytecode, so the assembly maps straight onto what the machine runs.

On top of that, pytoy adds a small C (minicc) and a fuller C with a real stack, so you can follow the whole transition: what happens when you write C, how it is translated down to bytecode, and how input/output works. (I/O for C is still being added.) The 256-address limit stays real and instructive

throughout: programs have to stay small, and it is easy to run out of room (see the note on memory limits below).

## 9.3 The machine

- **256 bytes of memory**, holding both program and data.
- **One accumulator** for arithmetic.
- **A program counter** that starts at 0.
- **14 opcodes**: load/store, add/sub, the bitwise operations, shifts, an unconditional `goto`, a single conditional `ifzero`, `nop`, and `stop`.

The original went through three versions: an experimental FreeDOS prototype, a Linux/ncurses prototype, and finally a FreeDOS graphics-mode program.

## 9.4 Building the original (and why v2 on macOS)

The three versions build very differently, which matters if you want to run Jim Hall's original alongside pytoy:

- **v3** (current `main`, the FreeDOS graphics-mode program) builds with a `.bat` script that calls **OpenWatcom C** (`wcl -q -2 -os toy.c …`). That toolchain targets DOS; it does not build on macOS out of the box.
- **v2** (the Linux/ncurses prototype) ships a plain **Makefile** using **gcc** and `-lncurses`. That is the version to check out on macOS — it builds with make + gcc and a terminal ncurses UI, no OpenWatcom or DOS needed. The screenshot above is this v2 build.

So on macOS: check out v2 to actually build and run the original. v3 is FreeDOS/OpenWatcom-only.

## 9.5 What comes from toycpu, and what is original

**Taken from toycpu (the instruction set):**

- The opcode values are identical: `STOP=0, RIGHT=1, LEFT=2, NOT=15, AND=17, OR=18, XOR=19, LOAD=20, STORE=21, ADD=22, SUB=23, GOTO=24, IFZERO=25, NOP=128`.
- The FETCH bit (bit 4, `0x10`) marking instructions that carry an address byte.
- The execution semantics of each instruction, including the 8-bit wraparound on ADD/SUB (`>255 #sym.arrow #sym.minus 256`, `<0 #sym.arrow #sym.plus 256`).

This is unavoidable: matching the instruction set *means* matching these numbers and their behaviour.

**Written from scratch for pytoy (not present in toycpu):**

- The two-pass **assembler** — labels, symbols, comments, data bytes. toycpu has no assembler; it reads pre-built binary via the switch-panel input shown above.
- The **Qt/PySide6 GUI debugger**. toycpu uses a DOS text-mode / ncurses display — a different language and toolkit entirely.
- The **CLI** verbose/step mode, the `EXPLAIN` output, the listing/export, and the `.toys`/`.toyo` file formats.

pytoy's Python is deliberately **simple, standard-library + PySide6 code** — readability is a feature, since the point is to *see* how the machine works.

## 9.6 Credits and license

The Toy CPU instruction set and design are by Jim Hall (FreeDOS Project). pytoy's own code (assembler, GUI, compiler) is independent.

Both projects are MIT-licensed: toycpu Copyright (c) 2022 Jim Hall (FreeDOS Project), pytoy Copyright (c) 2026 Manfred Scheucher. Jim Hall's original notice is reproduced in the repository's `NOTICE` file.